# Practical 7

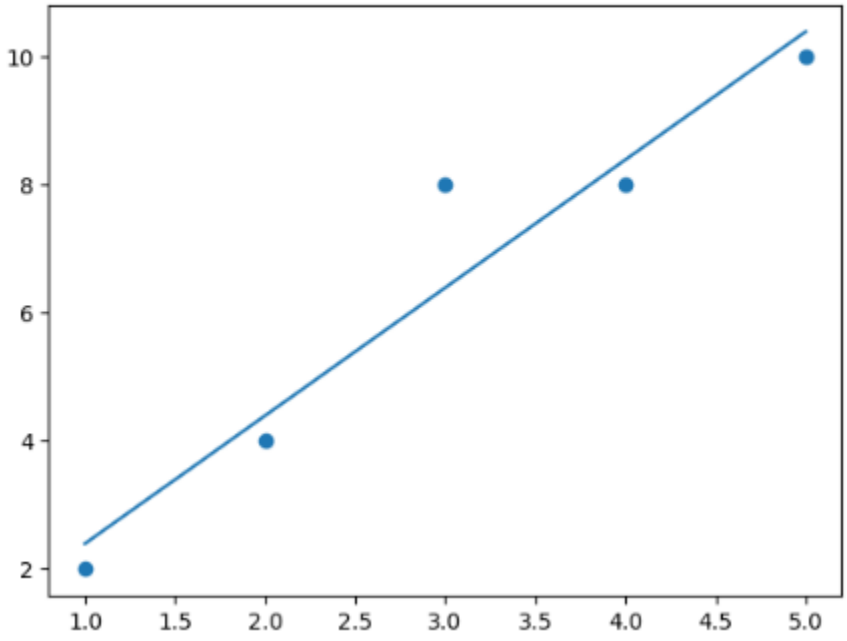
**7. For the given input values of x and y, compute coefficient values to build linear regression model and plot it. Use this model to predict the value of y for the new value of x. Also compute MSE and RSS. (Note: Use only numpy library)**

```
import numpy as np
import matplotlib.pyplot as plt

x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 8, 8, 10])

x_mean = np.mean(x)
y_mean = np.mean(y)

m = np.sum((x - x_mean)*(y - y_mean)) / np.sum((x - x_mean)**2)
c = y_mean - m*x_mean

print("Slope:", m)
print("Intercept:", c)

y_pred = m*x + c

mse = np.mean((y - y_pred)**2)
rss = np.sum((y - y_pred)**2)

print("MSE:", mse)
print("RSS:", rss)

x_new = 6
print("Predicted y for x=6:", m*x_new + c)

plt.scatter(x, y)
plt.plot(x, y_pred)
plt.show()
```

ET23BTIT107 Jalaj amit patel

**Output**

```
Slope: 2.0
Intercept: 0.40000000000000036
MSE: 0.6399999999999999
RSS: 3.1999999999999997
Predicted y for x=6: 12.4
```



# Practical 8

**8. For the given input values of x and y, build a linear regression model using scipy library and plot it. Also find co-relation coefficient (R-value), Probability value (P-value), and standard error. Try to modify the values in (X, Y) and identify its effect on**

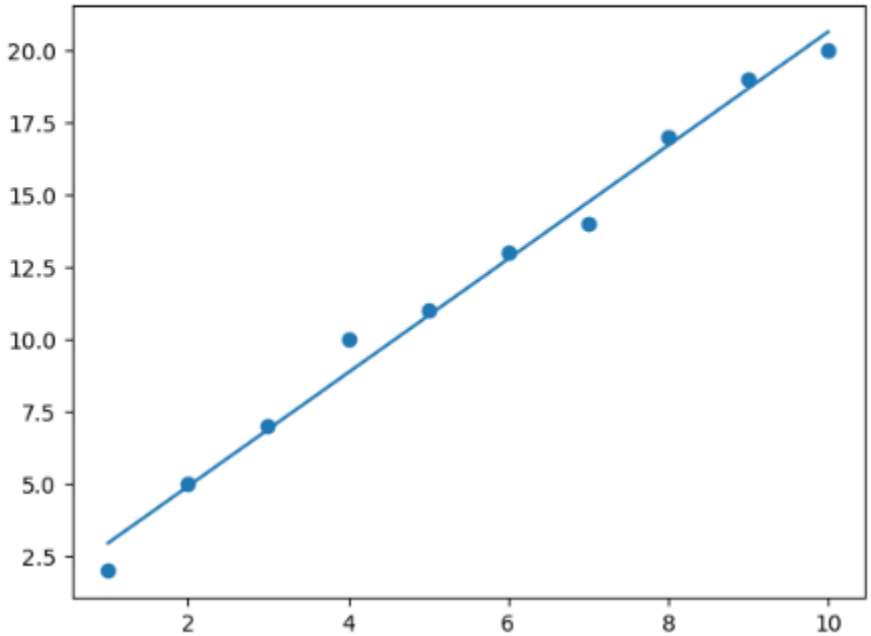
**the R-value, P-value and standard error.**

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import linregress

x = np.array([1,2,3,4,5,6,7,8,9,10])
y = np.array([2,5,7,10,11,13,14,17,19,20])

slope, intercept, r, p, std_err = linregress(x, y)

print("Slope:", slope)
print("Intercept:", intercept)
print("R-value:", r)
print("R-squared:", r**2)
print("P-value:", p)
print("Standard Error:", std_err)

plt.scatter(x, y)
plt.plot(x, slope*x + intercept)
plt.show()
```

ET23BTIT107 Jalaj amit patel

**Output:**

```
Slope: 1.9636363636363636
Intercept: 1.0
R-value: 0.994557782723072
R-squared: 0.9891451831750334
P-value: 3.812786621549432e-09
Standard Error: 0.07272727272727443
```



# Practical 9

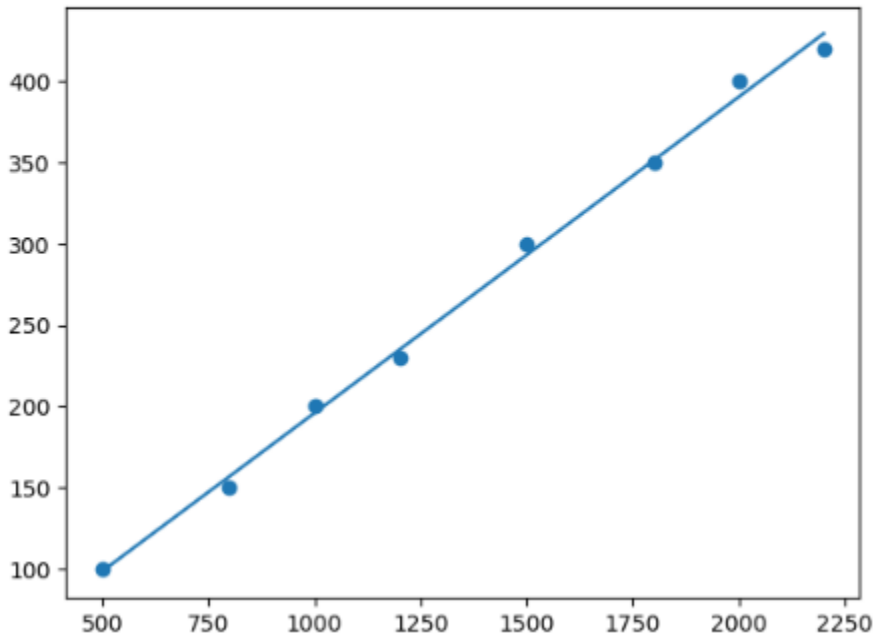
**9. Build a linear regression model that predicts house prices based on square footage(For your own dataset).**

```
import numpy as np
import matplotlib.pyplot as plt
from scipy.stats import linregress

sqft = np.array([500, 800, 1000, 1200, 1500, 1800, 2000, 2200])
price = np.array([100, 150, 200, 230, 300, 350, 400, 420])

slope, intercept, r, p, std_err = linregress(sqft, price)

print("Slope:", slope)
print("Intercept:", intercept)
print("R-value:", r)
print("P-value:", p)
print("Standard Error:", std_err)

new_sqft = 1600
print("Predicted Price:", slope*new_sqft + intercept)

plt.scatter(sqft, price)
plt.plot(sqft, slope*sqft + intercept)
plt.show()
```

ET23BTIT107 Jalaj amit patel

**Output:**

```
Slope: 0.1947731755424063
Intercept: 0.9368836291912999
R-value: 0.9983494886591722
P-value: 1.1226846522555927e-08
Standard Error: 0.0045742078848285544
Predicted Price: 312.5739644970414
```



# Practical 10

**10. Build a linear regression model using SKlearn library for the following :**

**a. Take input values in terms of lists (X,Y).**

**b. Generate training dataset and testing dataset from the input lists using**

**train_test_split() from SKlearn.**

**c. Train the model using training dataset.**

**d. Predict Y for test dataset.**

**e. Draw the plot.**

**f. Repeat step (b)-(e) for random pairs of (X,Y).**

```
import numpy as np

import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split

from sklearn.linear_model import LinearRegression



x = [5,7,8,7,2,17,2,9,4,11,12,9,6]

y = [99,86,87,88,111,86,103,87,94,78,77,85,86]



# convert list to array as LinearRegression model of SKlearn can work with array only

Train_x = np.array(x)

print("Train_x Before Reshaping: ", Train_x)
```

```
# Reshape X for sklearn

# To convert x list into one column vector

# reshape(-1,1) - No. of rows-unknown, no. of cols-1

Train_x = Train_x.reshape(-1,1)

print("Train_x After Reshaping: ", Train_x)


# Train model

Model = LinearRegression()

Model.fit(Train_x,y)


# use the same data for testing

Test_x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])

Test_y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])


print("Test_x Before Reshaping: ", Test_x)


# Reshape X for sklearn

Test_x = Test_x.reshape(-1,1)

print("Test_x After Reshaping: ", Test_x)


# Make predictions

y_pred = Model.predict(Test_x)

print(y_pred)
```

```
# Create visualization

plt.figure(figsize=(10, 6))

plt.scatter(Test_x, Test_y, color='blue', label='Test_x')

plt.plot(Test_x, y_pred, color='red', label='Regression Line')

plt.xlabel('X')

plt.ylabel('Y')

plt.title('X vs Y')

plt.legend()

plt.show()



# using train_test_split() and static data

x = np.array([5,7,8,7,2,17,2,9,4,11,12,9,6])

y = np.array([99,86,87,88,111,86,103,87,94,78,77,85,86])


X_train, X_test, y_train, y_test = train_test_split(

    x, y, test_size=0.2, random_state=42)


print(X_train)

print(X_test)


# Reshape X for sklearn
```

```
X_train = X_train.reshape(-1, 1)

y_train = y_train.reshape(-1, 1)

X_test = X_test.reshape(-1,1)


# Create and train the model

model = LinearRegression()

model.fit(X_train, y_train)


# Make predictions

y_pred = model.predict(X_test)


# Create visualization

plt.figure(figsize=(10, 6))

plt.scatter(X_test, y_test, color='blue', label='Test_x')

plt.plot(X_test, y_pred, color='red', label='Regression Line')

plt.xlabel('X')

plt.ylabel('Y')

plt.title('X vs Y')

plt.legend()

plt.show()



# using train_test_split() and dynamic data
```

```
# Set random seed for reproducibility
np.random.seed(42)

# Generate sample data
def generate_sample_data(n=100):
    xdata = np.random.uniform(1000, 5000, n)  # Uniform distribution
    print("No of Points = {}\nPoints = {}".format(xdata.size, xdata))

    # Normal distribution (bell-shaped)
    ydata = xdata * 100 + np.random.normal(0, 25000, n)
    return xdata, ydata

# Generate the data
X, y = generate_sample_data()

# Prepare datasets training-testing
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42)

print("\n\nTraining Dataset: {}\n\nTest Dataset: {}".format(X_train,
X_test))

# Reshape X for sklearn
```

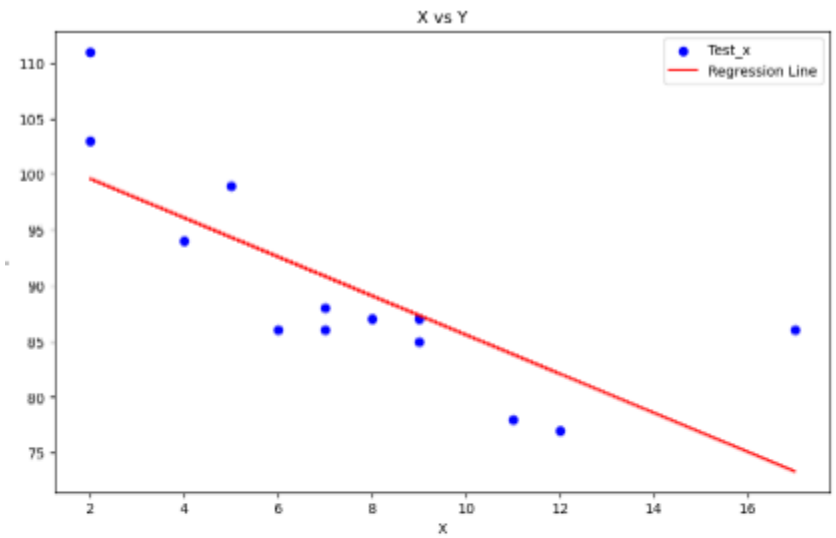
```
X_train = X_train.reshape(-1, 1)

y_train = y_train.reshape(-1, 1)

X_test = X_test.reshape(-1,1)


# Create and train the model

model = LinearRegression()

model.fit(X_train, y_train)


# Make predictions

y_pred = model.predict(X_test)


# Create visualization

plt.figure(figsize=(6, 6))

plt.scatter(X_test, y_test, color='blue', label='Test_x')

plt.plot(X_test, y_pred, color='red', label='Regression Line')

plt.xlabel('X')

plt.ylabel('Y')

plt.title('X vs Y for Random Data')

plt.legend()

plt.show()
```

ET23BTIT107 Jalaj amit patel

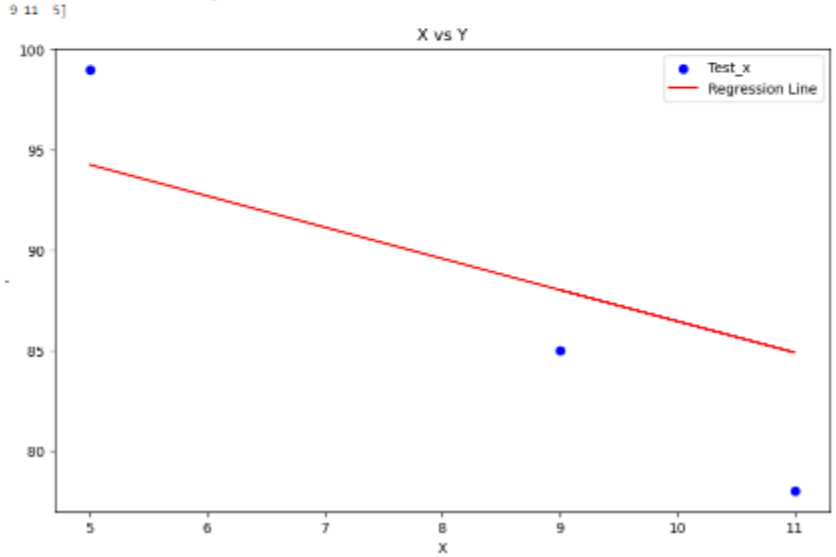
**Output:**

```
Train_x Before Reshaping:  [ 5  7  8  7  2 17  2  9  4 11 12  9  6]
Train_x After Reshaping:  [[ 5]
 [ 7]
 [ 8]
 [ 7]
 [ 2]
 [17]
 [ 2]
 [ 9]
 [ 4]
 [11]
 [12]
 [ 9]
 [ 6]]
Test_x Before Reshaping:  [ 5  7  8  7  2 17  2  9  4 11 12  9  6]
Test_x After Reshaping:  [[ 5]
 [ 7]
 [ 8]
 [ 7]
 [ 2]
 [17]
 [ 2]
 [ 9]
 [ 4]
 [11]
 [12]
 [ 9]
 [ 6]]
[94.34952171 90.84694628 89.09565857 90.84694628 99.60338484 73.33406917
 99.60338484 87.34437086 96.10080942 83.84179544 82.09050773 87.34437086
 92.598234  ]
```

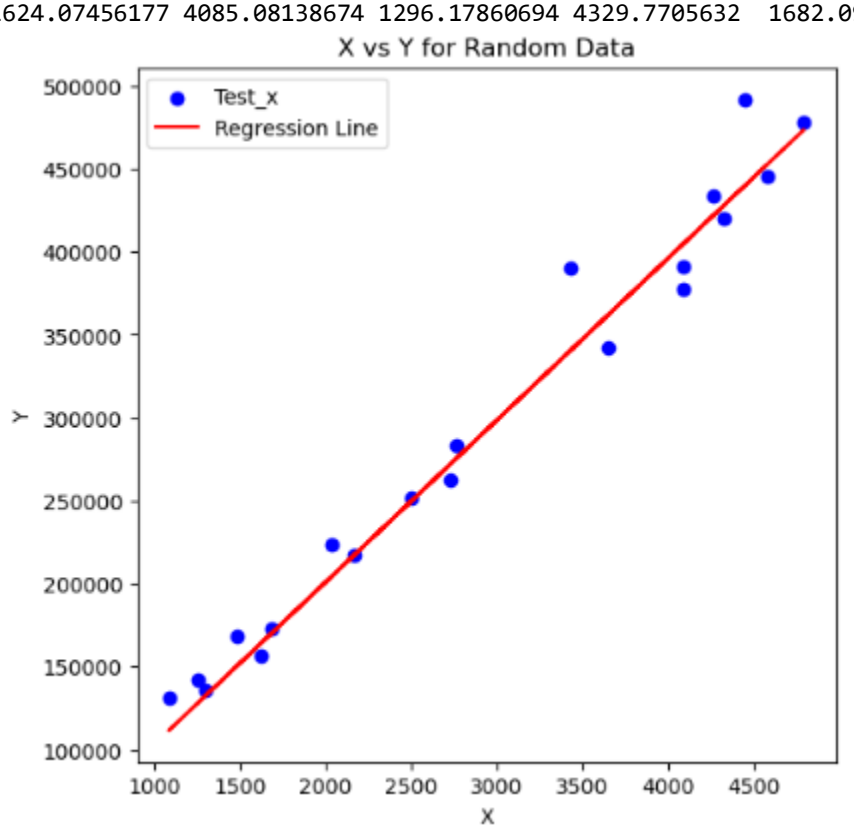
ET23BTIT107 Jalaj amit patel




4 17 8 7 6 2 9 12 7 2]
9 11 5]




```
No of Points = 100
Points = [2498.16047539 4802.85722564 3927.97576725 3394.63393679 1624.07456177
 1623.97808134 1232.33444867 4464.7045831  3404.46004697 3832.29031118
 1082.33797718 4879.63940865 4329.7705632  1849.35644271 1727.29986883
 1733.61803941 2216.96897184 3099.02572653 2727.78007457 2164.91656079
 3447.41157889 1557.97544261 2168.57859414 2465.44737317 2824.27993687
 4140.70384557 1798.69512863 3056.93775365 3369.65827545 1185.80165088
 3430.17940761 1682.09649475 1260.20637194 4795.54214901 4862.5281323
 4233.58939247 2218.45507669 1390.68845603 3736.93210605 2760.60997496
 1488.15293938 2980.70764045 1137.55408446 4637.28160832 2035.1199264
 3650.08913742 2246.84430436 3080.27208471 3186.84111737 1739.4178221
 4878.33851106 4100.53129344 4757.99576626 4579.30940171 3391.59991524
 4687.49694009 1353.97000821 1783.93144968 1180.90915564 2301.32132305
 2554.70915876 2085.3961271  4314.95003661 2427.01330677 2123.73803875
 3170.78433263 1563.6968999  4208.78792302 1298.20257472 4947.5477464
 4088.97907719 1794.86272614 1022.08846849 4261.84571382 3827.42937539
 3916.02867216 4085.08138674 1296.17860694 2433.86291418 1463.4762381
```

ET23BTIT107 Jalaj amit patel

```
 4452.4137035  3493.19250731 2323.59209941 1254.23340114 2243.92928686
 2300.73328811 3918.42471335 3550.22988542 4548.85097031 2888.85970065
 1478.37698375 3852.97914889 4043.14019447 3245.10879028 4083.86871982
 2975.18238546 3090.93131753 2710.16407343 1101.67650698 1431.56570797]


Training Dataset: [4687.49694009 4548.85097031 1798.69512863 1137.55408446 4947.54
77464
 1733.61803941 1488.15293938 3090.93131753 3832.29031118 1022.08846849
 4879.63940865 3080.27208471 2300.73328811 3369.65827545 3245.10879028
 1623.97808134 1563.6968999  3170.78433263 4233.58939247 2216.96897184
 1739.4178221  4862.5281323  4464.7045831  2975.18238546 3056.93775365
 2164.91656079 3493.19250731 4140.70384557 4314.95003661 1849.35644271
 2824.27993687 3394.63393679 3099.02572653 3736.93210605 3404.46004697
 2433.86291418 1232.33444867 2123.73803875 2218.45507669 2888.85970065
 1353.97000821 1431.56570797 3391.59991524 4637.28160832 4878.33851106
 4208.78792302 2246.84430436 1298.20257472 2085.3961271  2710.16407343
 1463.4762381  2980.70764045 1180.90915564 3186.84111737 1101.67650698
 1783.93144968 3916.02867216 1260.20637194 4083.86871982 2301.32132305
 2427.01330677 2243.92928686 1390.68845603 1185.80165088 4802.85722564
 4757.99576626 1557.97544261 3927.97576725 2465.44737317 3550.22988542
 3852.97914889 3827.42937539 3918.42471335 2323.59209941 3447.41157889
 2554.70915876 1794.86272614 1727.29986883 4043.14019447 4100.53129344]

Test Dataset: [1254.23340114 4579.30940171 4088.97907719 3650.08913742 2035.119926
4
 2760.60997496 2168.57859414 4452.4137035  1082.33797718 2498.16047539
 2727.78007457 3430.17940761 4261.84571382 4795.54214901 1478.37698375
 1624.07456177 4085.08138674 1296.17860694 4329.7705632  1682.09649475]
```